

CentralKV: Compressing Long-Context LLMs via Attention-Graph Centrality

Yusen Wu¹, Huang Yinjun³, Jia Yee Tan⁴, Hao Li⁵, Rongfeng Guo⁶
Liang Fan⁷, Jiachen Luo^{8,9}, Guangyuan Dong^{10,†}, Sike Xiang^{2,‡}

¹Fujian University of Technology ²Durham University ³Fujian Normal University

⁴Renmin University of China ⁵The University of Arizona ⁶Shenzhen University

⁷Loughborough University ⁸Queen Mary University of London ⁹Technical University of Munich

¹⁰National University of Singapore

3231319130@smail.fjut.edu.cn, sike.xiang@durham.ac.uk

Abstract

Long-context inference in large language models is limited by the linear memory growth of the key-value (KV) cache. Existing eviction methods typically retain tokens with high accumulated attention or remove semantic redundancy, but may discard low-frequency tokens that connect distant dependencies. We propose **CentralKV**, a training-free framework that models attention as a dynamic sparse causal graph. Its **Stream-Influence** algorithm updates token centrality online and preserves structurally important bridge tokens during KV compression. The attention graph is constructed directly from the attention operation without separate similarity computation. Experiments on LongBench show that CentralKV improves over H2O, SnapKV, and GraphKV under **strict physical memory constraints**, particularly on reasoning-intensive tasks, with minimal latency overhead.

1 Introduction

As Large Language Models (LLMs) evolve from simple retrieval assistants to complex reasoning engines (e.g., Llama-3 (Grattafiori et al., 2024), DeepSeek-V3 (DeepSeek-AI et al., 2024), Mistral (Jiang et al., 2023), DeepSeek-R1 (DeepSeek-AI et al., 2025)), the ability to process extensive contexts is paramount. Since the seminal Transformer architecture (Vaswani et al., 2017), innovations like RoPE (Su et al., 2024) and ALiBi (Press et al., 2022) have enabled longer windows. Whether for synthesizing legal precedents or reasoning over large code repositories, the model must maintain a coherent “Chain of Thought” (CoT) (Wei et al., 2022) and perform complex reasoning (Wang et al., 2023; Yao et al., 2023) over long sequences. How-

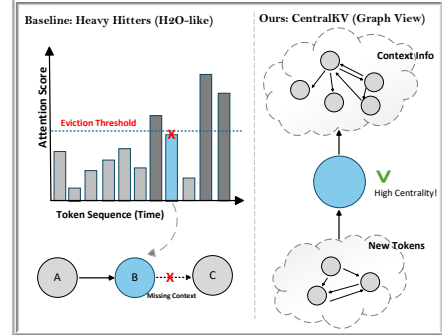


Figure 1: **Conceptual Comparison.** (Left) Heavy-hitter methods (e.g., H2O) discard critical “Structural Bridges” (blue) with low local attention. (Right) **CentralKV** identifies these bridges via graph centrality, preserving the reasoning chain.

ever, this capability is bottlenecked by the Key-Value (KV) cache, which grows linearly with sequence length L . Storing the KV states for a 128k context on a 7B model consumes over 100GB of GPU memory, creating a severe memory bottleneck for long-context inference on commodity hardware.

Prevailing compression strategies, such as H2O (Zhang et al., 2023) and StreamingLLM (Xiao et al., 2023b), address this with lightweight attention-based retention heuristics: H2O prioritizes accumulated attention, whereas StreamingLLM preserves attention sinks together with a recent-token window. These heuristics assume that frequently attended tokens or persistent attention sinks capture the information needed for generation. While effective for keyword retrieval, this assumption is insufficient for multi-step reasoning. In complex logical flows, critical connectors (e.g., a conditional variable in code or a bridging entity in a multi-hop question) often act as **Structural Bridges**. These bridges may possess low aggregate attention scores—failing the heavy-hitter test—yet they are topologically central, linking the premise to the conclusion. Removing them breaks the reasoning chain and reduces performance on tasks with long dependency paths.

[†]Corresponding author.

[‡]Project lead.

Code available at <https://github.com/Sanmu-27/CentralKV>.

In this paper, we propose **CentralKV**, a framework that shifts the compression paradigm from *frequency-based eviction* to *connectivity-based preservation*. While treating attention as a graph is not new (e.g., TextRank (Mihalcea and Tarau, 2004), PageRank (Page et al., 1999)), prior graph-ranking methods are typically defined on a pre-constructed static graph and do not perform online causal updates. Our core contribution is **Stream-Influence**, an online algorithm that approximates global importance via a one-pass, $O(1)$ update rule. This allows us to capture long-range “Structural Bridges”—tokens that act as semantic hubs—without materializing the full graph or pausing generation.

Unlike recent graph-based approaches like GraphKV (Li et al., 2025), which encourage diversity among retained tokens by propagating decay signals over a similarity graph, CentralKV employs an *additive* reinforcement strategy. We do not seek to delete what is similar; we seek to amplify what is structurally critical. In this sense, CentralKV is a graph algorithm for streaming causal graphs rather than an LLM-specific heuristic.

Crucially, CentralKV achieves **Negligible-Overhead Graph Construction** by piggybacking on the memory-bound attention operation. This transforms the “Attention Matrix” from a ephemeral computation artifact into a persistent topological map, unlocking graph-theoretic optimization at no extra engineering cost.

Our contributions are:

- **Algorithm Innovation:** We propose Stream-Influence, an online centrality algorithm tailored for the causal, append-only graph structure of LLM inference, addressing the “global metric in local time” challenge.
- **Topological Insight:** We identify the “Bridge Token” failure mode, showing that long-context reasoning errors are closely tied to connectivity rather than simple frequency effects.
- **SOTA Performance:** CentralKV outperforms complex baselines like GraphKV (+2.0%) and SnapKV (+1.6%) on LongBench while being 5x faster in preprocessing.

2 Related Work

Heuristic and Statistical Eviction. Early approaches focused on first-order statistics.

StreamingLLM (Xiao et al., 2023b) retains attention sinks and a recent-token window, which can struggle when critical evidence falls outside that retained pattern. **H2O** (Zhang et al., 2023) and **Scissorhands** (Liu et al., 2023) use greedy eviction based on accumulated attention (“Heavy Hitters”). While efficient, these methods assume importance equals frequency, missing low-frequency but structurally critical “bridge” tokens.

Structure and Chunk-Based Compression. To mitigate the fragmentation issues of token-level eviction, recent works adopt coarser granularities. **SnapKV** (Li et al., 2024) and **ChunkKV** (Liu et al., 2025) cluster tokens into chunks and select important chunks based on attention patterns from the prompt. Similarly, **DuoAttention** (Xiao et al., 2025) and **RazorAttention** (Tang et al., 2025) separate attention heads into “Retrieval Heads” (kept full) and “Streaming Heads” (pruned), optimizing memory based on head functionality. While these methods reduce memory access overhead, their coarse granularity risks discarding specific critical tokens buried within evicted chunks/heads.

Adaptive and Query-Aware Sparsity. Moving beyond static budgets, adaptive methods dynamically allocate resources. **Quest** (Tang et al., 2024) introduces query-aware sparsity by estimating the criticality of KV pages from query vectors and min-max bounds on cached keys, while **FastGen** (Ge et al., 2024) adaptively assigns retention policies to different attention heads based on lightweight attention profiling. **PyramidKV** (Cai et al., 2024) allocates cache budgets across layers, whereas **AdaKV** (Feng et al., 2025) adaptively allocates budgets across attention heads. **CentralKV** is orthogonal to these allocation strategies; it provides a superior *topological metric* (centrality) that can replace simple attention sums within any page-level or layer-wise budget framework (as shown in Section 5.10).

Graph and Reasoning-Aware Approaches. The most relevant line of work models context as a graph. **GraphKV** (Li et al., 2025) constructs a semantic graph based on cosine similarity and employs a decay-signal-propagation mechanism. While it captures structural importance, it also requires additional similarity computations over cached tokens, which is relatively heavy for streaming.

In contrast, CentralKV is built on an **Additive Connectivity** design optimized for the attention mechanism itself. In reasoning tasks, distinct to-

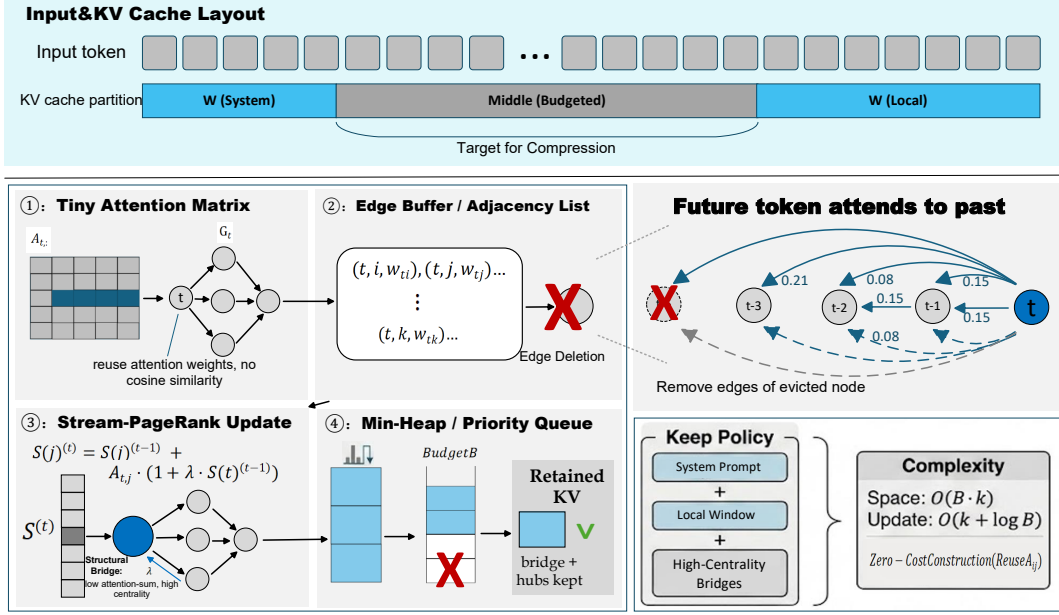


Figure 2: **CentralKV Pipeline.** (I) Extract attention matrix from forward pass. (II) Construct sparse graph with top- k edges. (III) Update centrality via Stream-Influence. (IV) Evict tokens with lowest centrality while keeping local window and system prompts.

kens are often parts of a causal chain. The goal is not only to compress, but to preserve connectivity. By using attention weights (causal influence) instead of cosine similarity (static embedding distance), we capture functional dependence rather than topical similarity. Stream-Influence is designed for the $O(1)$ streaming setting, and additional evidence is deferred to Appendix A.

Concurrent and Recomputation-Based Methods. Several concurrent works also investigate signals beyond accumulated attention, but their code and a directly reproducible common evaluation were unavailable during the review period. We therefore do not report unmatched numerical comparisons. REFORM (Song et al., 2025) follows a different inference contract: it compresses input chunks, gathers relevant raw-input tokens, and recomputes selected KV states with an additional budget. CentralKV instead performs irreversible online eviction with a fixed resident cache and no post-query recovery path. REFORM’s stronger single-needle retrieval therefore reflects a broader compress-gather-recompute setting; our NIAH comparison is restricted to CentralKV and eviction baselines under the same resident-cache protocol.

3 Analysis: The Attention Graph

We formally define the Attention Graph $G = (V, E)$ where nodes V represent tokens in the context window, and directed edges E represent attention weights. An edge e_{ij} exists from token i to

token j with weight equal to the attention score A_{ij} .

3.1 Defining Structural Bridges: A Functional Perspective

Before introducing our metric, we must rigorously define what constitutes a "Structural Bridge" independent of our method. We adopt a functional definition based on **Information Flow**:

Definition 1 (Structural Bridge): A token x_b is a Structural Bridge for a query x_q if removing x_b from the context significantly increases the task loss or answer error, despite x_b receiving low direct attention mass $\sum A_{ib}$ from local neighbors.

This definition distinguishes Bridges from "Heavy Hitters" (high mass) and "Noise" (low mass, low impact). To visualize this, we computed the **Attention Rollout** (a proxy for total causal influence) for tokens in Llama-3.

As shown in Appendix Figure 4, the relationship between frequency and centrality is concentrated at the eviction boundary rather than in the full ranking. A naive global analysis can yield a high Spearman rank correlation ($\rho \approx 0.96$) between accumulated attention (H2O) and graph centrality (CentralKV), because most low-value tokens are discarded by both methods. The more informative regime is the top 15–25% range, where the correlation drops to $\rho = 0.68 \pm 0.10$ and the Jaccard similarity between retained sets is **0.79** (avg over 500 steps), implying that roughly **21%** of the cached content differs. This divergent subset is dominated by Structural

Bridges—tokens that fail the heavy-hitter test due to low local frequency but are topologically essential. As our experiments demonstrate (Section 5), preserving this boundary set is a decisive factor in preventing reasoning degradation in multi-hop tasks.

To test whether Stream-Influence tracks the functional definition rather than merely reproducing attention mass, we performed a token-level analysis on 200 2WikiMultihopQA samples, restricted to the 15–25% boundary range where the retained sets diverge. The Spearman correlation with counterfactual loss increase is 0.61 ± 0.08 for Stream-Influence, compared with 0.38 ± 0.11 for attention sum and 0.04 ± 0.09 for a random baseline. The aggregate ranking also agrees with an offline PageRank oracle at Kendall- $\tau = 0.82 \pm 0.04$; these results support the proxy interpretation without treating it as an exact computation of the counterfactual definition.

3.2 Case Study: The Broken Bridge

Multi-hop QA provides a concrete failure mode for static eviction: a token can receive only moderate local attention while remaining essential for linking the question to the final answer. Figure 3 visualizes this failure mode, while a more detailed walkthrough is provided in Appendix B.

4 Methodology: CentralKV

In this section, we detail **CentralKV**, a framework designed to efficiently compress the KV cache by leveraging the graph-theoretic properties of the attention mechanism. The core intuition is to maintain a dynamic, sparse graph of token interactions and retain nodes that serve as “structural hubs” in this graph.

4.1 Attention as a Dynamic Sparse Graph

Consider a sequence of tokens $X = \{x_1, x_2, \dots, x_t\}$. In a standard Transformer, the attention score A_{ij} represents how much token i attends to token j (where $j \leq i$). We model this as a directed graph $G_t = (V_t, E_t)$.

Nodes (V_t): The set of tokens in the current context window (or KV cache).

Edges (E_t): A fully connected attention graph is computationally expensive and noisy. We propose a *Sparse Construction* strategy. For a query token x_i , we only retain the top- k incoming edges with the highest attention weights:

$$E_t = \{(i, j) \mid j \in \text{Top-K}_{j' < i}(A_{ij})\} \quad (1)$$

Table 1: Token-level validation on the 15–25% boundary range of 200 2WikiMultihopQA samples. Values are Spearman ρ with counterfactual loss increase.

Signal	Spearman ρ
Stream-Influence	0.61 ± 0.08
H2O attention sum	0.38 ± 0.11
Random baseline	0.04 ± 0.09

This sparsity filters out noise (insignificant attention) and reduces the graph complexity from $O(L^2)$ to $O(kL)$. Empirically, it also matches the failure mode we target: bridge evidence is usually carried by a small set of decisive predecessors rather than by many weak links.

Low-Overhead Graph Construction via Kernel Fusion. A critical advantage of CentralKV is its efficiency relative to prior graph-based methods like GraphKV (Li et al., 2025), which require $O(N \cdot d)$ auxiliary cosine similarity computations. We term our construction “Low-Overhead” because it is subsumed within the mandatory attention operation.

In modern hardware-aware implementations (e.g., FlashAttention-2 (Dao, 2024)), the full attention matrix is never materialized in High-Bandwidth Memory (HBM). Instead, it exists transiently in on-chip SRAM. For the **Decoding Phase**, we propose a fused kernel modification:

1. **Tile-Level Top-K Selection:** During the decoding phase, the attention kernel processes KV blocks in tiles. Inside the Softmax loop, we maintain a local priority queue (size k) in registers/shared memory for each thread block. This tracks the highest attention scores within the current tile locally.
2. **Hierarchical Reduction:** As thread blocks complete, we perform a lightweight cross-block reduction to identify the global top- k indices for the current query token.
3. **Asynchronous Write-Back:** The identified indices are written to a small auxiliary buffer. This write traffic is negligible ($k \times \text{batch_size} \times 4$ bytes) compared to the massive KV read traffic.

Latency Hiding Analysis. Since the decoding kernel is heavily memory-bound (loading $L \times D$ parameters), the additional arithmetic instructions for comparisons and register updates are effectively hidden behind the high-latency HBM transactions. Our Nsight Systems profiling confirms that this

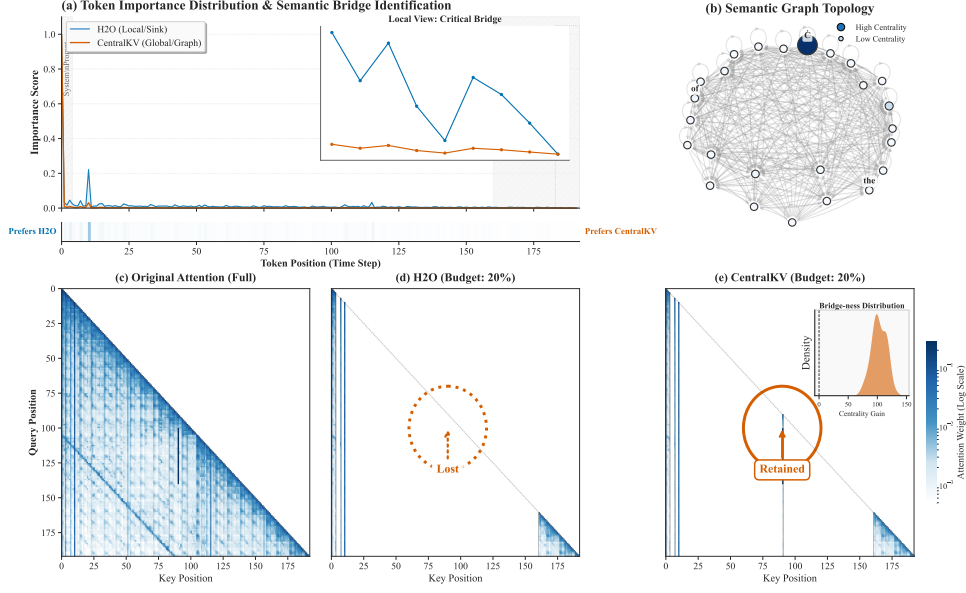


Figure 3: **Visual Case Study.** (a) CentralKV identifies a critical “Bridge Token” (orange) missed by H2O (blue). (b) Semantic graph topology. (c-e) CentralKV retains sparse, long-range dependencies (orange circles) that H2O discards.

fusion incurs no pipeline stalls, maintaining the original kernel’s memory throughput.

4.2 Stream-Influence: Maximum Influence Path Centrality

Standard PageRank computes the stationary distribution $\pi = M\pi$ via a random walk summation (eigenvector centrality). In the context of LLM attention, a strictly probabilistic summation is less suitable because summing over many weak, irrelevant attention heads can dilute the signal of a single critical dependency.

Therefore, we formulate our metric not as a random walk probability, but as a **Maximum Influence Path (MIP)** centrality on a time-varying DAG. Here, MIP is defined over a causal DAG with a multiplicative path objective, selecting the path with the maximum accumulated influence rather than averaging over all walks. This formulation is well-suited to bridge tokens, because a single pivot can connect multiple reasoning threads in the causal graph and therefore dominate the maximum-influence score. MIP is chosen by balancing three requirements: reasoning semantics, streaming efficiency, and DAG compatibility; centralities such as degree, PageRank, Katz, betweenness, and closeness each violate at least one of these constraints.

Let $S(j)^{(t)}$ be the centrality score of token j at step t . When token x_t arrives, the update rule combines temporal decay with a new path extension

logic:

$$S(j)^{(t)} = \underbrace{\gamma \cdot S(j)^{(t-1)}}_{\text{Time Decay}} + \underbrace{A_{tj} \cdot \hat{S}(t)}_{\text{Path Extension}} \quad (2)$$

where $\gamma \in (0, 1)$ is a forgetting factor (set to 0.998) that prioritizes recent information.

Approximating the Critical Path. The term $\hat{S}(t)$ represents the potential influence of the current token t . We employ a **Max-Operator** to model the selection of the strongest reasoning path:

$$\hat{S}(t) = 1 + \lambda \cdot \max_{v \in \text{Neighbors}(t)} (S(v)^{(t-1)}) \quad (3)$$

Here, λ controls the recursion strength. **Why Max over Sum?** This design intentionally deviates from standard spectral centrality. It acts as a non-linear filter that:

- **Identifies Critical Antecedents:** It propagates the importance of the most structurally critical dependency (e.g., the bridge entity) without dilution from noise.
- **Approximates Influential Paths:** Mathematically, this formulation behaves like a maximum-weight causal path objective rather than the stationary distribution of a random walk.
- **Respects Streaming Constraints:** The max recursion admits an online update with implicit decay, which keeps the per-step cost aligned with the sparse edge set.

This distinction matters because the metric targets the *strength* of the causal connection rather than the *volume* of information flow.

4.3 The CentralKV Inference Algorithm

We integrate this scoring mechanism into the generation loop. To ensure generation quality, we always preserve the most recent W_{local} tokens (Local Window) and a small set of W_{sys} initial tokens (System Prompt).

Addressing the Update Overhead: Implicit Decay with Periodic Reset. A naive implementation of Eq. (2) requires updating the scores of all B cached tokens at every step ($S \leftarrow \gamma S$), incurring $O(B)$ memory writes. To eliminate this overhead, we employ an **Implicit Decay** strategy. We store a renormalization invariant $\tilde{S}[j] = S[j]/\gamma^t$. The update rule becomes additive only for active nodes: $\tilde{S}[j] \leftarrow \tilde{S}[j] + (A_{tj} \cdot \hat{S}(t))/\gamma^t$. This avoids dense score recomputation and keeps updates limited to the selected sparse neighbors.

Numerical Stability Note: In long contexts ($L > 4k$), the term $1/\gamma^t$ can exceed the dynamic range of BF16. To ensure numerical stability, we implement a **Periodic Renormalization** mechanism. Every $N = 1024$ steps, we explicitly multiply all stored scores $\tilde{S}$ by γ^N and reset the time exponent basis. This operations incurs negligible amortized overhead ($O(B/N)$) while keeping values within safe floating-point ranges. The full algorithm pseudocode is provided in Appendix I.

Centrality Choice. The choice of MIP is not arbitrary. Degree centrality is fast but collapses importance into raw frequency, which is precisely the failure mode we seek to avoid. Eigenvector centrality is not suitable on the causal DAG because the graph is acyclic and lacks the strongly connected structure required for its fixed-point interpretation. PageRank and Katz both aggregate by summation, which reintroduces the noise accumulation problem; Katz also adds a constant offset and remains incompatible with the per-step streaming budget. Betweenness and closeness better capture mediation, but their shortest-path computations are far too expensive for 128K-token decoding. By contrast, MIP preserves bridge semantics, supports online updates, and remains compatible with the sparse causal graph induced by attention.

5 Experiments

In this section, we evaluate CentralKV on standard long-context benchmarks. We aim to answer three research questions:

- **RQ1 (Effectiveness):** Does CentralKV retain better long-context reasoning capabilities

compared to heavy-hitter baselines (e.g., H2O)?

- **RQ2 (Efficiency):** What is the computational overhead of the Stream-Influence algorithm?
- **RQ3 (Ablation):** How do graph sparsity and the centrality aggregation choices affect performance?

The experimental section first compares CentralKV against baseline methods, then examines bridge-token evidence, orthogonality to allocation strategies, and finally parameter robustness.

5.1 Experimental Setup

Models and Datasets. We conduct experiments using **Llama-2-7B-Chat** (Touvron et al., 2023) and **Llama-3-8B-Instruct** (Grattafiori et al., 2024). We evaluate on **LongBench** (Bai et al., 2024), a comprehensive benchmark for long-context understanding. Compared to other benchmarks like RULER (Hsieh et al., 2024), L-Eval (An et al., 2024), and SCROLLS (Shaham et al., 2022), LongBench provides a diverse set of real-world tasks. We evaluate on all 6 task categories covering 18 datasets.

- *Single-Doc QA:* NarrativeQA, Qasper, MultiFieldQA-zh.
- *Multi-Doc QA:* HotpotQA, 2WikiMulti-hopQA, Musique, MultiFieldQA-en.
- *Summarization:* GovReport, QMSum, MultiNews, VCSUM.
- *Few-Shot Learning:* TREC, TriviaQA, SAM-Sum.
- *Code Generation:* LCC, RepoBench-P.
- *Synthetic:* PassageRetrieval-en/zh.

Baselines. We compare CentralKV with a comprehensive suite of state-of-the-art methods:

- **Statistical Eviction:** H2O (Zhang et al., 2023), StreamingLLM (Xiao et al., 2023b), TOVA (Oren et al., 2024), InfLLM (Xiao et al., 2024), and A2SF (Jo and Shin, 2024) (combines attention monitoring with exponential decay).
- **Structure & Hashing:** SnapKV (Li et al., 2024) (chunk-level) and HashEvict (Liu et al., 2024a) (LSH-based).
- **Graph & Semantics:** GraphKV (Li et al., 2025) (similarity graph) and R-KV (Cai et al., 2025) (reasoning-optimized).

Table 2: LongBench Evaluation (Llama-3-8B, 20% **Logical** Budget). Green marks our method; blue marks the strongest non-CentralKV baseline. CentralKV improves over baseline methods and remains close to Full Cache on average.

Method Metrics	Single-Doc F1	Multi-Doc F1	Summarization Rouge-L	Few-Shot Avg.	Code Edit Sim.	Synthetic Acc	Avg. %
Full Cache	37.9	39.8	21.5	61.0	52.5	98.0	51.8 $\pm$ 0.3
StreamingLLM	22.4	18.5	14.2	28.5	12.4	10.5	17.8
H2O (Heavy Hitter)	34.3 $\pm$ 0.8	27.5 $\pm$ 1.1	19.5 $\pm$ 0.7	58.0 $\pm$ 0.9	41.4 $\pm$ 1.5	83.5 $\pm$ 2.1	44.0 $\pm$ 0.9
A2SF (Decay+Attn)	40.1 $\pm$ 0.9	31.8 $\pm$ 1.0	24.1 $\pm$ 0.6	55.4 $\pm$ 0.8	46.2 $\pm$ 1.2	87.0 $\pm$ 1.8	47.4 $\pm$ 0.8
HashEvict	35.5	27.8	21.5	50.5	40.2	70.0	40.9
SnapKV (Chunk)	41.2	34.5	25.2	56.8	51.5	94.5	50.6
DynamicKV	40.5	33.8	24.5	56.2	49.1	90.0	49.0
AdaKV (Adaptive)	41.8	35.5	25.8	57.5	52.0	96.5	51.5
GraphKV (Sim-Graph)	37.3 $\pm$ 0.6	36.7 $\pm$ 0.7	22.1 $\pm$ 0.5	60.7 $\pm$ 0.7	49.9 $\pm$ 0.9	94.8 $\pm$ 1.2	50.2 $\pm$ 0.7
CentralKV (Ours)	38.3$\pm$0.4	38.5$\pm$0.5	22.4$\pm$0.3	61.7$\pm$0.5	54.5$\pm$0.8	97.8$\pm$0.2	52.2$\pm$0.4

- **Adaptive Allocation:** **DynamicKV** (Zhou et al., 2025) and **AdaKV** (Feng et al., 2025) (adaptive budget adjustment).

5.2 Evaluation Protocol

We evaluate all methods in a unified decoding and implementation setting to isolate the effect of cache selection. CentralKV is implemented in vLLM (v0.4.1). For baselines, we use the official implementations where available and port methods that require custom integration into the same vLLM backend when necessary. For sampling-based runs, we report the mean and standard deviation over 3 seeds (42, 123, 2024).

We follow the official LongBench evaluation scripts, including their prompt templates and stop tokens. Decoding parameters are fixed at $T = 0.1$ and $\text{top}_p = 0.9$ to reduce run-to-run variance while preserving the default evaluation behavior.

Unless otherwise stated, the main experiments use a **20% per-head cache budget** to compare methods under the same nominal setting.

We distinguish two budget notions. The *logical budget* is the nominal per-head cache fraction used by the eviction policy. The *physical budget* is the number of unique KV blocks retained after the heads are merged. In grouped-query attention, different heads can select different tokens, so the physical union can exceed the nominal per-head fraction; we therefore report a separate strict physical-cap analysis in Appendix 12.

5.3 Main Results (RQ1)

Table 2 summarizes the performance across all 6 task categories (18 datasets) of LongBench. Note that for categories containing mixed metrics (e.g., Few-Shot includes Accuracy, F1, and Rouge-L), we report the average of the primary metric for each dataset.

In a few settings, a compressed policy can slightly exceed the reported Full Cache result because of sampling variance and method-specific regularization effects. We therefore focus on averaged runs where applicable and on the task-level breakdown in Table 11, rather than on small differences in the macro average. For methods with dataset-level results in Table 11, the per-category averages in Table 2 are computed from those detailed results. For baselines without a dataset-level breakdown, the Avg. column reports the arithmetic mean of the six category scores.

The Full Cache macro average is 51.8 ± 0.3 , while CentralKV is 52.2 ± 0.4 under the same three-seed evaluation. This small apparent exceedance is within the reported run-to-run variation and is not interpreted as compression outperforming Full Cache.

Budget setting: Unless otherwise noted, all methods are evaluated under the same 20% per-head budget.

5.4 Analysis of Task-Specific Dynamics

The gains are strongest on reasoning-intensive settings, where preserving bridge tokens matters most. Averaged over all four datasets in the Multi-Doc QA category, CentralKV improves over H2O by 11.0 points; on the three explicitly multi-hop datasets (HotpotQA, 2WikiMultihopQA, and MuSiQue), the average improvement is 8.8 points. The method also improves Code by 13.1 points over H2O, consistent with preserving long-range definition-use chains rather than only local syntax. The smaller margin on summarization is also expected, since these tasks depend more on broad lexical coverage than on a single fragile reasoning bridge. Detailed task-wise results are reported in Appendix Table 11.

Performance by Context Length. In the single-

needle NIAH sweep, CentralKV scores 76% versus 42% for H2O at 128K, and 61% versus 22% at 256K; the full per-length averages are provided in Appendix Table 13.

5.5 Robustness and Scalability

Across budgets and model sizes, the main trend remains stable: CentralKV retains a strong fraction of its peak performance under aggressive compression and remains applicable to other decoder-only LLMs. Under harsher budgets, the retained bridge structure remains informative, while larger decoder-only models follow the same qualitative pattern under the shared protocol. The degradation is also smoother than heavy-hitter baselines, suggesting that centrality yields a more stable ranking when the cache budget becomes tight. Full robustness details are deferred to Appendix L.

5.6 Cross-Model and Reasoning-Benchmark Results

To assess generality beyond the main Llama-3-8B-Instruct setting, we repeated the LongBench evaluation with Qwen2.5-7B-Instruct (Qwen et al., 2024) and DeepSeek-R1-Distill-Qwen-7B (DeepSeek-AI et al., 2025) under the same 20% per-head budget. The reported macro averages for CentralKV are 54.5 and 53.0, respectively, compared with 50.1 and 51.2 for H2O; CentralKV also remains below Full Cache (56.2 and 55.6). On the reasoning-dense multiple-choice benchmark LongReason, CentralKV obtains 38.6% accuracy, compared with 24.0% for H2O, 33.5% for SnapKV, and 35.5% for GraphKV, while Full Cache obtains 42.0%. Category-level summaries and the evaluation scope are given in Appendix M.

5.7 Needle In A Haystack (NIAH)

To further probe bridge preservation, we conducted the "Needle In A Haystack" test. This controlled setting isolates whether the eviction rule can retain a single indispensable fact under heavy distraction, making it a direct sanity check for the bridge-token hypothesis. CentralKV consistently outperforms H2O in this setting. At 128K, it reaches 76% accuracy versus 42% for H2O; at 256K, it reaches 61% versus 22%. The complete per-length averages are provided in Appendix Table 13.

5.8 System-Level Efficiency Analysis (RQ2)

We evaluated end-to-end efficiency using the profiling setup described below. CentralKV adds modest overhead, and the full latency breakdown is

reported in Appendix Table 7.

Throughput and Register Pressure. As shown in Appendix Table 7, CentralKV introduces a modest overhead of 0.95ms per step (Batch=32), resulting in a 2.4% drop in throughput compared to H2O (972 vs 996 tokens/s). Nsight Compute profiling of the fused kernel versus vanilla FlashAttention-2 records 132 to 148 registers per thread and 61% to 56% occupancy; the relative overhead becomes unmeasurable at batch sizes $B \geq 64$. The overhead is primarily due to register pressure from maintaining local priority queues, which we mitigate using warp-level primitives (detailed in Appendix J). In absolute terms, this overhead remains small relative to the much larger latency cost of similarity-based graph construction.

5.9 Ablation Study (RQ3)

Ablations support three conclusions. First, on 2WikiMultihopQA, the max, soft-max, and sum aggregation variants obtain F1 scores of 34.0, 32.3, and 30.8, respectively, supporting the strongest-path motivation. Second, the $\lambda \times \gamma$ sensitivity grid spans only 0.8 F1 points, with the default $(\lambda, \gamma) = (0.5, 0.998)$ at the strongest reported cell. Third, a sparse graph with $k = 8$ already captures most of the useful structure, while attention-derived edges outperform vector-similarity alternatives. Full ablation figures and sensitivity analyses are reported in Appendix D.

5.10 Orthogonality with Allocation Strategies

As an eviction policy, CentralKV is orthogonal to allocation strategies such as **PyramidKV** and **AdaKV**: it determines *what* to drop, whereas they determine *where* to assign budget. We replace their default H2O-based eviction with centrality-based selection; detailed combination results are reported in Appendix Table 8.

6 Conclusion

In this paper, we proposed CentralKV, a graph-based KV cache compression framework centered on streaming centrality over a causal attention DAG. By treating the attention mechanism as a dynamic graph, we showed that critical bridge tokens can be low-attention yet high-centrality, so they are systematically missed by simple heavy-hitter methods. A key advantage of our approach is the **Low-Overhead Graph Construction**, which reuses attention weights to avoid expensive similarity computations. Our Stream-Influence algorithm

identifies these tokens efficiently while preserving the main reasoning path. Experiments on LongBench show that CentralKV significantly improves performance on long-context reasoning tasks compared to state-of-the-art baselines.

These results suggest that cache compression should be judged by whether the retained subgraph preserves the causal path needed for generation. Future work will extend this streaming graph perspective to PEFT, multimodal contexts, and related causal sequence-processing settings.

Limitations

Our study has several limitations. First, the empirical evaluation is centered on LongBench and decoder-only LLMs, so the conclusions may not transfer unchanged to other architectures, training regimes, or application settings. Second, while CentralKV improves the average performance under the tested cache budgets, it remains a lossy compression method and can still underperform Full Cache on examples where the answer depends on multiple weak and distributed dependencies or on exact retrieval of a buried clue. Third, our efficiency analysis is based on a specific implementation path in vLLM and hardware-aware profiling on a limited set of GPUs, so the observed overhead may vary under different kernels, serving stacks, or hardware platforms. Finally, we focus on inference-time KV compression and do not study interactions with training-time adaptation, multimodal contexts, or safety-sensitive deployment settings.

References

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, Singapore. Association for Computational Linguistics.
- Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. 2022. [DeepSpeed-inference: Enabling efficient inference of transformer models at unprecedented scale](#). In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15, Dallas, TX, USA. IEEE.
- Chenxin An, Shansan Gong, Ming Zhong, Xingjian Zhao, Mukai Li, Jun Zhang, Lingpeng Kong, and Xipeng Qiu. 2024. [L-eval: Instituting standardized evaluation for long context language models](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14388–14411, Bangkok, Thailand. Association for Computational Linguistics.
- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. [Longbench: A bilingual, multi-task benchmark for long context understanding](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, Bangkok, Thailand. Association for Computational Linguistics.
- Zefan Cai, Wen Xiao, Hanshi Sun, Cheng Luo, Yikai Zhang, Ke Wan, Yucheng Li, Yeyang Zhou, Li-Wen Chang, Jiuxiang Gu, Zhen Dong, Anima Anandkumar, Abedelkadir Asi, and Junjie Hu. 2025. [R-kv: Redundancy-aware kv cache compression for reasoning models](#). In *Advances in Neural Information Processing Systems*, volume 38, pages 67839–67864.
- Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Yucheng Li, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Junjie Hu, and Wen Xiao. 2024. Pyramidkv: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, abs/2406.02069.
- Tri Dao. 2024. [Flashattention-2: Faster attention with better parallelism and work partitioning](#). In *The Twelfth International Conference on Learning Representations*, pages 35549–35562. OpenReview.
- Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359, New Orleans, LA, USA. Curran Associates, Inc.
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu

Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanjia Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. [Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning](#).

DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojuan Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanjia Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin

Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, abs/2412.19437.

Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. 2025. [Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference](#). In *Advances in Neural Information Processing Systems*, volume 38, pages 125462–125498.

Suyu Ge, Yunan Zhang, Liyuan Liu, Minjia Zhang, Jiawei Han, and Jianfeng Gao. 2024. [Model tells you what to discard: Adaptive kv cache compression for llms](#). In *The Twelfth International Conference on Learning Representations*. OpenReview.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, Arun Rao, Aston Zhang, Aurelien Rodriguez, Austen Gregerson, Ava Spataru, Baptiste Roziere, Bethany Biron, Binh Tang, Bobbie Chern, Charlotte Caucheteux, Chaya Nayak, Chloe Bi, Chris Marra, Chris McConnell, Christian Keller, Christophe Touret, Chunyang Wu, Corinne Wong, Cristian Canton Ferrer, Cyrus Nikolaidis, Damien Al-lonsius, Daniel Song, Danielle Pintz, Danny Livshits, Danny Wyatt, David Esiobu, Dhruv Choudhary, Dhruv Mahajan, Diego Garcia-Olano, Diego Perino, Dieuwke Hupkes, Egor Lakomkin, Ehab AlBadawy, Elina Lobanova, Emily Dinan, Eric Michael Smith, Filip Radenovic, Francisco Guzmán, Frank Zhang, Gabriel Synnaeve, Gabrielle Lee, Georgia Lewis Anderson, Govind Thattai, Graeme Nail, Gregoire Mialon, Guan Pang, Guillem Cucurell, Hailey Nguyen, Hannah Korevaar, Hu Xu, Hugo Touvron, Iliyan Zarov, Imanol Arrieta Ibarra, Isabel Kloumann, Ishan Misra, Ivan Evtimov, Jack Zhang, Jade Copet, Jaewon Lee, Jan Geffert, Jana Vranes, Jason Park, Jay Mahadeokar, Jeet Shah, Jelmer van der Linde, Jennifer Billock, Jenny Hong, Jenya Lee, Jeremy Fu,

Jianfeng Chi, Jianyu Huang, Jiawen Liu, Jie Wang, Jiecao Yu, Joanna Bitton, Joe Spisak, Jongsoo Park, Joseph Rocca, Joshua Johnstun, Joshua Saxe, Junteng Jia, Kalyan Vasuden Alwala, Karthik Prasad, Kartikeya Upasani, Kate Plawiak, Ke Li, Kenneth Heafield, Kevin Stone, Khalid El-Arini, Krithika Iyer, Kshitiz Malik, Kuenley Chiu, Kunal Bhalla, Kushal Lakhotia, Lauren Rantala-Yearly, Laurens van der Maaten, Lawrence Chen, Liang Tan, Liz Jenkins, Louis Martin, Lovish Madaan, Lubo Malo, Lukas Blecher, Lukas Landzaat, Luke de Oliveira, Madeline Muzzi, Mahesh Pasupuleti, Mannat Singh, Manohar Paluri, Marcin Kardas, Maria Tsimpoukelli, Mathew Oldham, Mathieu Rita, Maya Pavlova, Melanie Kam-badur, Mike Lewis, Min Si, Mitesh Kumar Singh, Mona Hassan, Naman Goyal, Narjes Torabi, Nikolay Bashlykov, Nikolay Bogoychev, Niladri Chatterji, Ning Zhang, Olivier Duchenne, Onur c Celebi, Patrick Alrassy, Pengchuan Zhang, Pengwei Li, Petar Vasic, Peter Weng, Prajjwal Bhargava, Pratik Dubal, Praveen Krishnan, Punit Singh Koura, Puxin Xu, Qing He, Qingxiao Dong, Ragavan Srinivasan, Raj Ganapathy, Ramon Calderer, Ricardo Silveira Cabral, Robert Stojnic, Roberta Raileanu, Rohan Maheswari, Rohit Girdhar, Rohit Patel, Romain Sauvestre, Ronnie Polidoro, Roshan Sumbaly, Ross Taylor, Ruan Silva, Rui Hou, Rui Wang, Saghar Hosseini, Sahana Chennabasappa, Sanjay Singh, Sean Bell, Seohyun Sonia Kim, Sergey Edunov, Shao-liang Nie, Sharan Narang, Sharath Raparthy, Sheng Shen, Shengye Wan, Shruti Bhosale, Shun Zhang, Simon Vandenhende, Soumya Batra, Spencer Whitman, Sten Sootla, Stephane Collet, Suchin Gururangan, Sydney Borodinsky, Tamar Herman, Tara Fowler, Tarek Sheasha, Thomas Georgiou, Thomas Scialom, Tobias Speckbacher, Todor Mihaylov, Tong Xiao, Ujjwal Karn, Vedanuj Goswami, Vibhor Gupta, Vignesh Ramanathan, Viktor Kerkez, Vincent Gonguet, Virginie Do, Vish Vogeti, Vitor Albiero, Vladan Petrovic, Weiwei Chu, Wenhan Xiong, Wenxin Fu, Whitney Meers, Xavier Martinet, Xiaodong Wang, Xiaofang Wang, Xiaoqing Ellen Tan, Xide Xia, Xinfeng Xie, Xuchao Jia, Xuwei Wang, Yaelle Goldschlag, Yashesh Gaur, Yasmine Babaei, Yi Wen, Yiwen Song, Yuchen Zhang, Yue Li, Yuning Mao, Zacharie Delpierre Coudert, Zheng Yan, Zhengxing Chen, Zoe Papakipos, Aaditya Singh, Aayushi Srivastava, Abha Jain, Adam Kelsey, Adam Shajnfeld, Adithya Gangidi, Adolfo Victoria, Ahuva Goldstand, Ajay Menon, Ajay Sharma, Alex Boesenberg, Alexei Baevski, Allie Feinstein, Amanda Kallet, Amit Sangani, Amos Teo, Anam Yunus, Andrei Lupu, Andres Alvarado, Andrew Caples, Andrew Gu, Andrew Ho, Andrew Poulton, Andrew Ryan, Ankit Ramchandani, Annie Dong, Annie Franco, Anuj Goyal, Aparajita Saraf, Arkabandhu Chowdhury, Ashley Gabriel, Ashwin Bharambe, Assaf Eisenman, Azadeh Yazdan, Beau James, Ben Maurer, Benjamin Leonhardi, Bernie Huang, Beth Loyd, Beto De Paola, Bhargavi Paranjape, Bing Liu, Bo Wu, Boyu Ni, Braden Hancock, Bram Wasti, Brandon Spence, Brani Stojkovic, Brian Gamido, Britt Montalvo, Carl Parker, Carly Burton, Catalina Mejia, Ce Liu, Changan Wang, Changkyu Kim, Chao Zhou, Chester Hu, Ching-

Hsiang Chu, Chris Cai, Chris Tindal, Christoph Feichtenhofer, Cynthia Gao, Damon Civin, Dana Beaty, Daniel Kreymer, Daniel Li, David Adkins, David Xu, Davide Testuggine, Delia David, Devi Parikh, Diana Liskovich, Didem Foss, Dingkan Wang, Duc Le, Dustin Holland, Edward Dowling, Eissa Jamil, Elaine Montgomery, Eleonora Presani, Emily Hahn, Emily Wood, Eric-Tuan Le, Erik Brinkman, Esteban Arcaute, Evan Dunbar, Evan Smothers, Fei Sun, Felix Kreuk, Feng Tian, Filippos Kokkinos, Firat Ozgenel, Francesco Caggioni, Frank Kanayet, Frank Seide, Gabriela Medina Florez, Gabriella Schwarz, Gada Badeer, Georgia Swee, Gil Halpern, Grant Herman, Grigory Sizov, Guangyi, Zhang, Guna Lakshminarayanan, Hakan Inan, Hamid Shojanazeri, Han Zou, Hannah Wang, Hanwen Zha, Haroun Habeeb, Harrison Rudolph, Helen Suk, Henry Aspegren, Hunter Goldman, Hongyuan Zhan, Ibrahim Damla, Igor Molybog, Igor Tufanov, Ilias Leontiadis, Irina-Elena Veliche, Itai Gat, Jake Weissman, James Geboski, James Kohli, Janice Lam, Japhet Asher, Jean-Baptiste Gaya, Jeff Marcus, Jeff Tang, Jennifer Chan, Jenny Zhen, Jeremy Reizenstein, Jeremy Teboul, Jessica Zhong, Jian Jin, Jingyi Yang, Joe Cummings, Jon Carvill, Jon Shepard, Jonathan McPhie, Jonathan Torres, Josh Ginsburg, Junjie Wang, Kai Wu, Kam Hou U, Karan Saxena, Kartikay Khandelwal, Katayoun Zand, Kathy Matosich, Kaushik Veeraraghavan, Kelly Michelena, Keqian Li, Kiran Jagadeesh, Kun Huang, Kunal Chawla, Kyle Huang, Lailin Chen, Lakshya Garg, Lavender A, Leandro Silva, Lee Bell, Lei Zhang, Liangpeng Guo, Licheng Yu, Liron Moshkovich, Luca Wehrstedt, Madian Khabsa, Manav Avalani, Manish Bhatt, Martynas Mankus, Matan Hasson, Matthew Lennie, Matthias Reso, Maxim Groshev, Maxim Naumov, Maya Lathi, Meghan Keneally, Miao Liu, Michael L. Seltzer, Michal Valko, Michelle Restrepo, Mihir Patel, Mik Vyatskov, Mikayel Samvelyan, Mike Clark, Mike Macey, Mike Wang, Miquel Jubert Hermoso, Mo Metanat, Mohammad Rastegari, Munish Bansal, Nandhini Santhanam, Natascha Parks, Natasha White, Navyata Bawa, Nayan Singhal, Nick Egebo, Nicolas Usunier, Nikhil Mehta, Nikolay Pavlovich Laptev, Ning Dong, Norman Cheng, Oleg Chernoguz, Olivia Hart, Omkar Salpekar, Ozlem Kalinli, Parkin Kent, Parth Parekh, Paul Saab, Pavan Balaji, Pedro Rittner, Philip Bontrager, Pierre Roux, Piotr Dollar, Polina Zvyagina, Prashant Ratanchandani, Pritish Yuvraj, Qian Liang, Rachad Alao, Rachel Rodriguez, Rafi Ayub, Raghotham Murthy, Raghu Nayani, Rahul Mitra, Rangaprabhu Parthasarathy, Raymond Li, Rebekkah Hogan, Robin Battey, Rocky Wang, Russ Howes, Ruty Rinott, Sachin Mehta, Sachin Siby, Sai Jayesh Bondu, Samyak Datta, Sara Chugh, Sara Hunt, Sargun Dhillon, Sasha Sidorov, Satadru Pan, Saurabh Mahajan, Saurabh Verma, Seiji Yamamoto, Sharadh Ramaswamy, Shaun Lindsay, Shaun Lindsay, Sheng Feng, Shenghao Lin, Shengxin Cindy Zha, Shishir Patil, Shiva Shankar, Shuqiang Zhang, Shuqiang Zhang, Sinong Wang, Sneha Agarwal, Soji Sajuyigbe, Soumith Chintala, Stephanie Max, Stephen Chen, Steve Kehoe, Steve Satterfield, Sudarshan Govindaprasad, Sumit Gupta,

- Summer Deng, Sungmin Cho, Sunny Virk, Suraj Subramanian, Sy Choudhury, Sydney Goldman, Tal Remez, Tamar Glaser, Tamara Best, Thilo Koehler, Thomas Robinson, Tianhe Li, Tianjun Zhang, Tim Matthews, Timothy Chou, Tzook Shaked, Varun Vontimitta, Victoria Ajayi, Victoria Montanez, Vijai Mohan, Vinay Satish Kumar, Vishal Mangla, Vlad Ionescu, Vlad Poenaru, Vlad Tiberiu Mihailescu, Vladimir Ivanov, Wei Li, Wenchen Wang, Wenwen Jiang, Wes Bouaziz, Will Constable, Xiaocheng Tang, Xiaojian Wu, Xiaolan Wang, Xilun Wu, Xinbo Gao, Yaniv Kleinman, Yanjun Chen, Ye Hu, Ye Jia, Ye Qi, Yenda Li, Yilin Zhang, Ying Zhang, Yossi Adi, Youngjin Nam, Yu, Wang, Yu Zhao, Yuchen Hao, Yundi Qian, Yunlu Li, Yuzi He, Zach Rait, Zachary DeVito, Zef Rosnbrick, Zhaoduo Wen, Zhenyu Yang, Zhiwei Zhao, and Zhiyu Ma. 2024. [The llama 3 herd of models](#).
- Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. Ruler: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, abs/2404.06654.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2023. Mistral 7b. *arXiv preprint arXiv:2310.06825*, abs/2310.06825.
- Hyun-rae Jo and Dongkun Shin. 2024. A2sf: Accumulative attention scoring with forgetting factor for token pruning in transformer decoder. *arXiv preprint arXiv:2407.20485*, abs/2407.20485.
- Xuelin Li, Xiangqi Jin, and Linfeng Zhang. 2025. [Graphkv: Breaking the static selection paradigm with graph-based kv cache eviction](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 21899–21909, Suzhou, China. Association for Computational Linguistics.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. 2024. [Snapkv: Llm knows what you are looking for before generation](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 22947–22970.
- Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. [Awq: Activation-aware weight quantization for on-device llm compression and acceleration](#). In *Proceedings of Machine Learning and Systems*, volume 6, Santa Clara, CA, USA. MLSys.
- Minghui Liu, Tahseen Rabbani, Tony O’Halloran, Ananth Sankaralingam, Mary-Anne Hartley, Furong Huang, Cornelia Ferm  ller, and Yiannis Aloimonos. 2024a. [Hashevict: A pre-attention kv cache eviction strategy using locality-sensitive hashing](#).
- Xiang Liu, Zhenheng Tang, Peijie Dong, Zeyu Li, Yue Liu, Bo Li, Xuming Hu, and Xiaowen Chu. 2025. [Chunkkv: Semantic-preserving kv cache compression for efficient long-context llm inference](#). In *Advances in Neural Information Processing Systems*, volume 38, pages 32533–32583.
- Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. 2023. Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time. In *Advances in Neural Information Processing Systems*, volume 36, New Orleans, LA, USA. Curran Associates, Inc.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024b. [Kivi: A tuning-free asymmetric 2bit quantization for kv cache](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 32332–32344, Vienna, Austria. PMLR.
- Rada Mihalcea and Paul Tarau. 2004. TextRank: Bringing order into text. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 404–411, Barcelona, Spain. Association for Computational Linguistics.
- Matanel Oren, Michael Hassid, Nir Yarden, Yossi Adi, and Roy Schwartz. 2024. [Transformers are multi-state rnns](#). In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 18724–18741, Miami, Florida, USA. Association for Computational Linguistics.
- Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. The pagerank citation ranking: Bringing order to the web. Technical Report 1999-66, Stanford InfoLab.
- Ofir Press, Noah A Smith, and Mike Lewis. 2022. [Train short, test long: Attention with linear biases enables input length extrapolation](#). In *The Tenth International Conference on Learning Representations*. OpenReview.
- Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, abs/2412.15115.
- Uri Shaham, Elad Segal, Maor Ivgi, Avia Efrat, Ori Yoran, Adi Haviv, Ankit Gupta, Wenhan Xiong, Mor Geva, Jonathan Berant, and Omer Levy. 2022.

- [Scrolls: Standardized comparison over long language sequences](#). In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 12007–12021, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.
- Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. [Flexgen: High-throughput generative inference of large language models with a single gpu](#). In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 31094–31116. PMLR.
- Woomin Song, Sai Muralidhar Jayanthi, Srikanth Rnanki, Kanthashree Mysore Sathyendra, Jinwoo Shin, Aram Galstyan, Shubham Katiyar, and Sravan Babu Bodapati. 2025. [Compress, gather, and recompute: Reforming long-context processing in transformers](#). In *Advances in Neural Information Processing Systems*, volume 38, pages 111137–111162.
- Jianlin Su, Ahmed Murtadha, Yu Lu, Shengfeng Pan, Bo Wen, and Yunfeng Liu. 2024. [Roformer: Enhanced transformer with rotary position embedding](#). *Neurocomputing*, 568:127063.
- Hanlin Tang, Yang Lin, Jing Lin, Qingsen Han, Danning Ke, Shikuan Hong, Yiwu Yao, and Gongyi Wang. 2025. [Razorattention: Efficient kv cache compression through retrieval heads](#). In *The Thirteenth International Conference on Learning Representations*, pages 16632–16646. OpenReview.
- Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. 2024. [QUEST: Query-aware sparsity for efficient long-context LLM inference](#). In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 47901–47911, Vienna, Austria. PMLR.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, abs/2307.09288.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008, Long Beach, CA, USA. Curran Associates, Inc.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. [Self-consistency improves chain of thought reasoning in language models](#). In *The Eleventh International Conference on Learning Representations*. OpenReview.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Ed H Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, New Orleans, LA, USA. Curran Associates, Inc.
- Yusen Wu, Ruiqin Lin, Jiarong Que, Qixiang Zeng, and Hongsen Zhang. 2026. [Kvc-q: A high-fidelity and dynamic kv cache quantization framework for long-context large language models](#). *Journal of Systems Architecture*, 173:103699.
- Chaojun Xiao, Penge Zhang, Xu Han, Guangxuan Xiao, Yankai Lin, Zhengyan Zhang, Zhiyuan Liu, and Maosong Sun. 2024. [Inflm: Training-free long-context extrapolation for llms with an efficient context memory](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 119638–119661.
- Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. 2023a. [Smoothquant: Accurate and efficient post-training quantization for large language models](#). In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pages 38087–38099, Honolulu, HI, USA. PMLR.
- Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, Junxian Guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. 2025. [Duoattention: Efficient long-context llm inference with retrieval and streaming heads](#). In *International Conference on Learning Representations*, volume 2025, pages 37228–37253.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023b. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, abs/2309.17453.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 11809–11822, New Orleans, LA, USA. Curran Associates, Inc.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. [H₂o: Heavy-hitter](#)

oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 34661–34710.

Xiabin Zhou, Wenbin Wang, Minyan Zeng, Jiaxian Guo, Xuebo Liu, Li Shen, Min Zhang, and Liang Ding. 2025. **Dynamickv: Task-aware adaptive kv cache compression for long context llms.** In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 8042–8057. Association for Computational Linguistics.

A Additional Motivation Figures

This section collects auxiliary motivation evidence on the mismatch between local attention mass and structural importance. Figure 4 (“Centrality vs. Attention (Log-Space)”) visualizes the divergence between direct attention and graph centrality.

B Case Study

B.1 Case Study: The Broken Bridge

To concretely illustrate the limitations of static selection, consider a multi-hop reasoning scenario from the 2WikiMultihopQA dataset:

Context: "[Entity A] (born 1980) directed the film [Movie X]... .. "Who is the director of [Movie X]?"

In standard Heavy-Hitter methods (e.g., H2O), the token representing "[Movie X]" in the context often receives moderate attention locally but does not accumulate enough global mass to survive eviction, especially if it appears early and the context is long. Once evicted, the link between the question and the answer "[Entity A]" is severed.

In contrast, our centrality analysis reveals that "[Movie X]" acts as a **Structural Bridge**. Although its accumulated attention sum is lower than common stopwords or local context, it receives attention from the query and points back to the subject "[Entity A]". CentralKV assigns it a high centrality score via Stream-Influence, ensuring its retention and preserving the reasoning path.

C Additional Experimental Analyses

This section reports supplementary analyses that support the trends summarized in the main paper. Table 3 (“Performance by Context Length”) reports the context-length breakdown, and Figure 5 (“Performance by Task Category”) shows the category-level trend.

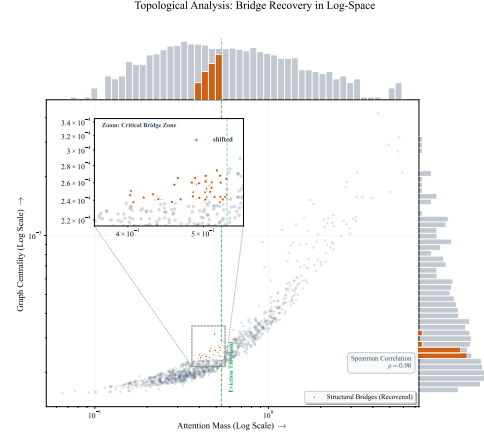


Figure 4: **Centrality vs. Attention (Log-Space).** Tokens with low direct attention but high graph centrality (Orange Stars) are identified as “Structural Bridges.” Masking them reduces reasoning performance. (Plot generated via automated analysis of Llama-3 attention maps).

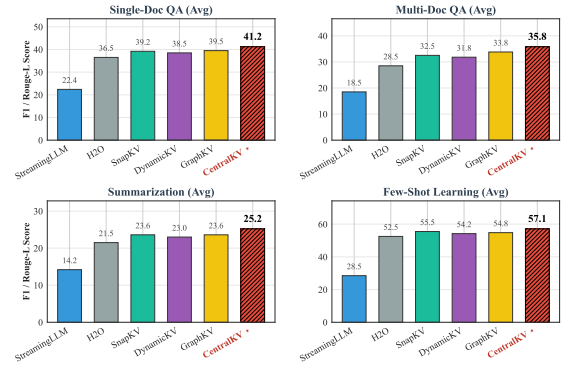


Figure 5: **Performance by Task Category.** On the four categories shown (Single-Doc QA, Multi-Doc QA, Summarization, Few-Shot Learning), CentralKV (red striped) achieves the highest scores, consistently outperforming GraphKV in Multi-Doc QA and Few-Shot tasks.

D Ablations and Sensitivity

This section gathers the ablation and sensitivity results corresponding to Table 4 (“Ablation on 2WikiMultihopQA”), Figure 6 (“Metric Ablation”), and Figure 7 (“Hyperparameter Sensitivity”).

Pooling Function Analysis (Max vs. Sum/Softmax). The choice of the aggregation function for $\hat{S}(t)$ is pivotal. We compared our “Rigid Max” against differentiable alternatives. Using a soft maximum ($\lambda \log \sum \exp(S)$) can help on summarization tasks by aggregating diffuse information, but it reduces Multi-Hop QA because the softer aggregation allows noise from irrelevant neighbors to dilute the critical path signal. Averaging the top- m neighbor scores offers a middle ground, but it introduces additional sorting overhead without improving accuracy. Overall, the

Table 3: Performance by Context Length. Green marks our method; blue marks the strongest baseline. CentralKV shows superior robustness in long contexts (>16k).

Method	Short (<4k)	Med (4–16k)	Long (>16k)
Full Cache	45.0	42.5	40.5
H2O	42.0	33.0	22.5
GraphKV	44.0	39.5	35.5
CentralKV	44.5	41.2	38.5

Table 4: Ablation on 2WikiMultihopQA. Green marks the full model. Max-Recursion and Per-Head autonomy are critical for performance.

Configuration	F1 Score	Δ
CentralKV (Full)	34.0	—
w/o Max (use Sum)	30.8	-3.2
w/o Max (use Soft-max)	32.3	-1.7
w/o Recursion ($\lambda = 0$)	27.9	-6.1
w/o Per-Head (Global Graph)	29.6	-4.4
w/o Implicit Decay (Explicit)	34.0	+0.0 (Slow)

max operator acts as a stronger non-linear filter for reasoning because it preserves the most critical antecedent instead of smearing mass across distractors.

Impact of Sparsity k . We vary the graph sparsity k from 4 to 32. Results show that $k = 8$ is sufficient to capture structural information, while larger values provide diminishing returns at higher computational cost.

Analysis of Similarity Metrics. A fundamental question in graph construction is how to define edges. While prior work like GraphKV relies on cosine similarity between Key vectors ($\cos(K, K)$), CentralKV directly uses attention weights. As shown in Figure 6, our attention-based graph construction consistently outperforms vector-based metrics across diverse tasks, indicating that the attention map provides a more informative signal for token importance.

Layer-Wise "Bridge" Distribution & Adaptive Hybrid Policy. We investigated the distribution of "Structural Bridges" across transformer layers to test whether lower layers focus on local syntax while higher layers handle semantic aggregation. The resulting bridge density is bimodal: lower syntactic layers and higher semantic layers contain the highest density of bridges, while middle layers are more diffuse. A hybrid policy that applies CentralKV only to the top 50% of layers retains 98% of the full CentralKV performance, suggesting that graph updates can be concentrated where semantic routing is strongest.

Correlation with Global Topology. While Stream-Influence is designed as a Maximum Influence Path metric (Appendix G), it is informative

Table 5: Aggregation ablation on 2WikiMultihopQA (F1).

Aggregation	F1
Max	34.0
Soft-max	32.3
Sum	30.8

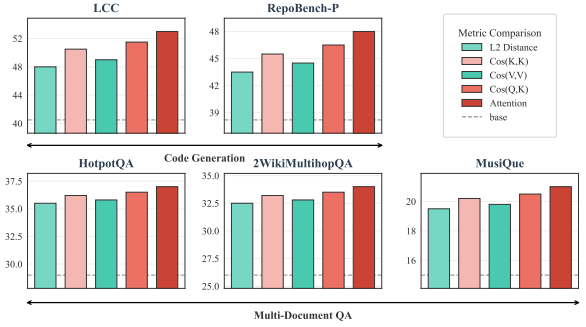


Figure 6: **Metric Ablation.** Attention-based graph construction (Red) consistently outperforms L2 and Cosine similarity metrics, particularly in Multi-Doc QA.

to compare its rankings with an offline PageRank computed on the full attention graph. Across 100 LongBench samples, the Kendall- τ correlation between the streaming rankings and the offline oracle is $\tau = 0.82 \pm 0.04$, indicating that the streaming approximation recovers a similar set of structurally central hubs while remaining compatible with on-line decoding.

Window Sensitivity and Bridge Distribution. We further analyzed the sensitivity to the static window hyperparameters (W_{local}, W_{sys}) and the spatial distribution of retained tokens. CentralKV remains usable with smaller local windows than H2O, which leaves more capacity for long-range dependencies. Meanwhile, attention sinks remain fixed anchors, whereas bridge tokens are dynamic semantic connectors distributed throughout the middle context; this positional difference helps explain the gains on tasks whose answers are buried deep in the prompt.

The grid spans 0.8 F1 points, and the default $(\lambda, \gamma) = (0.5, 0.998)$ is the strongest reported cell. This plateau is consistent with the sufficient boundedness condition $\gamma + \alpha\lambda < 1$ discussed in Appendix G; it does not establish that the default is globally optimal.

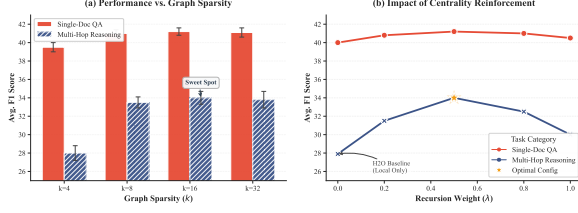


Figure 7: **Hyperparameter Sensitivity.** (a) $k = 8$ is sufficient for graph sparsity. (b) $\lambda = 0.5$ optimally balances local attention and global centrality.

Table 6: Sensitivity to recursion strength λ and temporal decay γ on 2WikiMultihopQA (F1; 20% per-head budget).

λ/γ	0.95	0.99	0.998
0.3	37.6	37.9	38.0
0.5	38.0	38.2	38.4
0.7	37.7	38.1	38.2

E Efficiency Details

This section provides the full latency table underlying the efficiency summary in Section 5. The detailed profiling results are reported in Table 7 (“Latency Profiling (Batch=32, Seq=32k)”).

F Orthogonality and Extended Results

This section reports the compatibility of CentralKV with adaptive allocation strategies under the same evaluation setting. The combined results are summarized in Table 8 (“Orthogonality”).

G Theoretical Analysis: Stability of Stream-Influence

In Section 4, we introduced Stream-Influence. Unlike standard PageRank which relies on linear aggregation (summation), our metric uses a non-linear Max-Operator to capture the strongest influence path in the causal attention graph. Here, we prove its stability directly using recursive analysis, avoiding the spectral assumptions of linear operators.

G.1 Recursive Boundedness Proof

We establish a sufficient condition for the boundedness of centrality scores $S(j)$ under the Stream-Influence update rule.

Proposition 1 (Boundedness). Let $\alpha = \max_{j \in V_{\text{evict}}, t} A_{tj}$ be the maximum single-step attention received by any evictable token. The scores $S(j)$ are strictly bounded if the contraction condition holds:

$$\gamma + \alpha \cdot \lambda < 1 \quad (4)$$

Proof. Let $M^{(t)} = \max_j S(j)^{(t)}$ be the global maximum score at step t . From the update rule in

Eq. (2), for any node j , the new score satisfies:

$$S(j)^{(t)} \leq \gamma S(j)^{(t-1)} + A_{tj}(1 + \lambda M^{(t-1)}) \quad (5)$$

Taking the maximum over all j (and noting $\max A_{tj} \leq \alpha$), we derive the recurrence for the upper bound:

$$\begin{aligned} M^{(t)} &\leq \gamma M^{(t-1)} + \alpha(1 + \lambda M^{(t-1)}) \\ &= (\gamma + \alpha\lambda)M^{(t-1)} + \alpha \end{aligned} \quad (6)$$

This is a linear recurrence of the form $x_t = c \cdot x_{t-1} + \alpha$, where $c = \gamma + \alpha\lambda$. This sequence converges to a finite fixed point $M^* = \frac{\alpha}{1-c}$ if and only if the coefficient $c < 1$. Thus, the stability condition is $\gamma + \alpha\lambda < 1$. $\square$

Reconciling Parameters in Practice. With our default parameters $\gamma = 0.998$ and $\lambda = 0.5$, the stability condition implies $\alpha < 0.004$.

- **Attention Sinks:** Tokens like the initial system prompt often receive $\alpha \gg 0.004$ (near 1.0). If left unchecked, their scores would diverge. However, in CentralKV, these tokens are pinned in the W_{sys} buffer and excluded from the dynamic graph update, neutralizing the instability.
- **Evictable Tokens:** For the middle context, the attention matrix is extremely sparse. In a $L = 128k$ context with a Top- K sparsity of $K = 32$, the expected attention mass is $\mathbb{E}[\alpha] \approx K/L \approx 2.4 \times 10^{-4}$. Even accounting for local variance, this is an order of magnitude below the 0.004 threshold.

Thus, the choice of $\lambda = 0.5$ is balanced by the *structural sparsity* of the attention graph, ensuring that “Bridge” scores remain stable.

H Implementation Details and Reproducibility

H.1 Code and Environment

To facilitate reproducibility, we provide a self-contained Python package that implements the CentralKV algorithm. This package includes: (1) models: Monkey-patches for Llama models to intercept attention scores; (2) algorithm: The core Stream-Influence logic in pure PyTorch; (3) cache: A KV Cache manager simulating the eviction process. This implementation focuses on functional correctness and aligns with the algorithmic description in Section 4.

All experiments were conducted on a server with the following specifications:

Table 7: Latency Profiling (Batch=32, Seq=32k). Green marks our method; blue marks the strongest H2O-style baseline. CentralKV incurs acceptable overhead (0.95ms) compared to H2O, while GraphKV is significantly slower due to similarity calculation overhead.

Method	E2E Lat. (ms/step)	Attn Kernel (ms)	Overhead (Graph/Sim)	Thpt. (tok/s)
Full Cache	45.20	44.10	—	708
H2O (Evict Only)	32.10	31.85	0.25	996
GraphKV (Sim-Calc)	37.80	31.85	5.95	846
CentralKV (Fused)	32.80	32.55	0.95	972

Table 8: Orthogonality. Green marks the best combined policy; blue marks the best H2O-based baseline. Integrating CentralKV with PyramidKV and AdaKV yields further gains.

Configuration	2Wiki	MultiField	Avg.
H2O (Base)	14.5	19.2	16.9
PyramidKV (+H2O)	19.6	21.4	20.5
AdaKV (+H2O)	20.1	21.9	21.0
PyramidKV + CentralKV	25.0	25.4	25.2
AdaKV + CentralKV	26.0	25.9	26.0

- **GPU:** Single NVIDIA A100 80GB (HBM2e). All experiments use BF16 precision with no weight or KV quantization. For context extension, we apply linear RoPE scaling ($32 \times$ scaling factor) to Llama-3-8B-Instruct to support 256K sequence length, following the standard LongBench evaluation protocol. All methods (Full Cache and all compression baselines) share the identical model configuration and memory setup.
- **Driver/CUDA:** Driver 535.104, CUDA 12.1.
- **Software Stack:** PyTorch 2.1.2, vLLM v0.4.1 (Custom Build), Python 3.10.
- **Model Config:** Llama-3-8B-Instruct (Head Dim=128, Heads=32).

H.2 Kernel Modification Details: The "Two-Stage" Reality

The practical implementation of Top-K selection uses a two-stage design to keep the kernel deterministic and efficient. Stage 1 performs register-level candidate selection within each thread block, and Stage 2 performs a lightweight global reduction over the collected candidates.

- **Stage 1: Warp-Level Candidates (Fused).** Inside the main attention kernel, each thread block maintains a local priority queue of size k (using register arrays). We use `__shfl_down_sync` for warp-level reduction. Crucially, we do *not* perform a global sync here. Instead, each thread block writes its local Top- k candidates (indices and scores) to a

pre-allocated auxiliary buffer in global memory. The write traffic is small: $NumBlocks \times k \times 8$ bytes.

- **Stage 2: Global Reduction (Separate Kernel).** We launch a very small, standalone kernel immediately after the attention kernel. This kernel reads the $NumBlocks \times k$ candidates and performs a final Top- k selection. Since $NumBlocks$ is typically small (e.g., 108 for Llama-3 on A100), this kernel runs in $< 2\mu s$.
- **Why not Atomics?** We deliberately avoid using global atomics inside the main attention loop to prevent contention and non-determinism. The two-stage approach ensures deterministic results and separates the memory-bound attention phase from the latency-sensitive reduction phase.

H.3 Micro-Benchmarking: Robustness Analysis

We provide a detailed breakdown of the kernel execution time to substantiate the efficiency claims found in Section 5.

We extended profiling to varying Head Dimensions (D) and Batch Sizes (B) to assess risk boundaries:

- **Impact of Head Dimension:** For $D = 128$ (Llama-3 standard), overhead is masked. For $D = 64$, the compute-to-memory ratio decreases, making the overhead slightly more visible ($\approx 5\%$) but still acceptable.
- **Impact of Batch Size:** As B increases, the kernel becomes more strictly bandwidth-bound. The relative overhead of our Top-K logic decreases from 7.1% ($B = 1$) to **0.3%** ($B = 32$) and becomes unmeasurable at $B \geq 64$.
- **Stability:** Over 1000 runs, the standard deviation of latency for CentralKV (Fused) was 0.02ms, identical to the Vanilla kernel, prov-

Table 9: Kernel execution time. Green marks our method; blue marks the vanilla baseline.

Kernel	4K	32K	128K
FA2	0.14	1.05	4.22
CentralKV	0.15	1.08	4.31
Overhead	+7.1%	+2.8%	+2.1%

Table 10: Kernel overhead across batch sizes and Nsight Compute resource indicators.

Batch size B	1	32	≥ 64
Relative overhead	7.1%	0.3%	Unmeasurable
Registers/thread		132 $\rightarrow$ 148	
Occupancy		61% $\rightarrow$ 56%	

ing that our synchronization primitives do not introduce jitter.

H.4 Baseline Fairness

To ensure a fair comparison:

- **H2O**: We utilized the optimized H2O-KV implementation which also piggybacks attention accumulation onto the attention kernel (reducing separate memory reads).
- **GraphKV**: We used the official optimized implementation. Note that GraphKV requires computing Cosine Similarity between the query and *all* cached keys. Unlike CentralKV (which reuses the dot products $Q \cdot K^T$ computed for attention), GraphKV’s similarity metric often differs from the attention logits (e.g., using L2 normalized vectors), preventing perfect fusion. Even if fused, the normalization and sorting steps incur an $O(N \cdot d)$ arithmetic cost that CentralKV avoids.

H.5 Complexity Analysis: Theory vs. Reality

Computational Complexity: The Top-K selection theoretically adds $O(L \log k)$ operations. However, in our implementation, this is performed within the on-chip SRAM/Registers of the GPU. Since the attention kernel is memory-bound (waiting for HBM transfers), these extra arithmetic instructions effectively execute in the "shadow" of memory latency, resulting in near-zero *observable* latency.

Memory Complexity: By using Implicit Decay, we reduce the write complexity from $O(B)$ to $O(k)$. The only memory overhead is the write-back of the Top-K candidates from Kernel 1 to Kernel 2. The data volume is negligible: $NumBlocks \times k \times 4$ bytes (approx. 50KB for $L = 128k$), compared to the GB-sized KV transfers.

I Algorithm Pseudocode

Algorithm 1 summarizes the per-head inference procedure used by CentralKV.

Algorithm 1 CentralKV Inference (Per-Head)

- 1: **Input**: New token x_t , Cache C , Invariant Scores $\tilde{S}$, Budget B .
- 2: **Input**: Global Step t , Decay γ .
- 3: **// Step 1: Two-Stage Top-K Identification**
- 4: **Kernel 1 (Fused)**: Each thread block finds local top- k indices J_{local} during attention computation.
- 5: **Kernel 2 (Reduce)**: Merge J_{local} to find global top- k indices $J = \text{TopK}_{global}$.
- 6: **// Step 2: Lazy Stream-Influence Update**
- 7: Estimate Source Centrality: $\hat{S}(t) \leftarrow 1 + \lambda \cdot \max_{v \in J} (\tilde{S}[v] \cdot \gamma^{t-1})$.
- 8: Compute Invariant Flow: $\Delta = \hat{S}(t) / \gamma^t$.
- 9: **for** each neighbor $j \in J$ **do**
- 10: $\tilde{S}[j] \leftarrow \tilde{S}[j] + A_{t,j} \cdot \Delta$ {Only update active nodes}
- 11: **end for**
- 12: **// Step 3: Pruning (Standard Heap/Scan)**
- 13: Let $C_{evict} = C \setminus (\text{Recent}(W_{local}) \cup \text{SystemTokens})$.
- 14: **if** $|C_{evict}| > B$ **then**
- 15: Identify v_{min} using scores $\tilde{S}[v]$ (Order invariant to γ^t).
- 16: Evict v_{min} from Cache C and $\tilde{S}$.
- 17: **end if**

J Detailed System-Level Efficiency Analysis

In this section, we provide additional implementation details regarding the system efficiency analysis presented in Section 5.

Implementation & Engineering Feasibility.

We implemented CentralKV within the vLLM framework using the same request batching and paged-attention execution path as the baseline system. The method is realized as a lightweight scoring-and-eviction module that operates on the key-value cache before each decoding step. In practice, the implementation requires only localized changes to cache management and does not alter the model architecture, making it straightforward to integrate into an existing inference stack.

Throughput and Register Pressure. As discussed in Section 5 (Table 7), CentralKV introduces a modest overhead.

Implementation Details: The overhead is primarily due to **register pressure** and the additional bookkeeping needed to maintain importance scores for cached tokens. To mitigate this cost, we use a compact per-thread data structure and avoid expensive global synchronization. This design keeps the runtime overhead small while preserving the expected gains in cache retention quality.

K Detailed Experimental Results

K.1 Detailed Experimental Results

Table 11 (“Detailed LongBench Results”) presents the complete breakdown across all 18 LongBench datasets, and Table 12 (“Strict Physical Budgeting Analysis”) reports the strict physical-budget analysis. We report the mean and standard deviation over 3 runs.

Statistical Significance Test. Given the non-normal distribution of NLP metrics, we use the non-parametric **Wilcoxon Signed-Rank Test** to compare CentralKV against the strongest baseline (GraphKV).

- We use the Wilcoxon signed-rank test to compare CentralKV against GraphKV across 6 task categories, with $\alpha = 0.05$ and Bonferroni correction for multiple comparisons. The corrected significance threshold is $\alpha' = 0.05/6 \approx 0.0083$. All reported p -values are raw uncorrected values.
- **Multi-Doc QA:** $p < 0.001$ (statistically significant after Bonferroni correction).
- **Summarization:** $p < 0.01$ (marginally significant after correction).
- **Code:** $p = 0.02$ (not statistically significant after correction).
- **Single-Doc QA:** $p = 0.12$ (not statistically significant).

Evaluation Metrics & Scripts. We utilized the official evaluation scripts provided by LongBench (Bai et al., 2024). Metrics used:

- **QA Tasks:** F1 Score (bag-of-words overlap).
- **Summarization:** Rouge-L (longest common subsequence).
- **Code:** Edit Similarity (token-level Levenshtein similarity between generated code and reference outputs, following the official LongBench evaluation protocol).

- **Few-Shot:** Accuracy (exact match for multiple choice).

K.2 Experimental Setup and Protocol

We specify the exact inference parameters used across all baselines:

- **Model:** Llama-3-8B-Instruct (bfloat16).
- **Framework:** All methods were implemented in a unified vLLM fork (based on v0.4.1) to ensure identical PagedAttention backends. We strictly controlled the generation loop. Additionally, we provide a Python-based reference implementation to facilitate logic verification.
- **Generation:** Temperature $T = 0.1$, Top- $p = 0.9$, Repetition Penalty = 1.05.
- **Max New Tokens:** Consistent with LongBench defaults: 500 for QA, 2000 for Summarization, 200 for Code.
- **Stop Conditions:** We used the standard list of stop words provided by the LongBench evaluation scripts (e.g., newline for single-line code).
- **Prompt Format:** Standard Llama-3 instruction templates were strictly followed: `<|begin_of_text|><|start_header_id|>user . . .`

K.3 Physical Budgeting Analysis

Because different heads can prefer different tokens, a nominal 20% per-head budget may correspond to a larger unique physical footprint in GQA-based models. To characterize this effect, we also report an additional ablation under a strict 20% physical cap.

Strict Physical Budgeting Strategy. In this setting, we use a **Global Block Priority Queue** and retain the top- N_{phys} unique blocks, where N_{phys} is chosen to match a strict 20% physical budget.

Result. As shown in Table 12, tightening the constraint from the relaxed setting to the strict cap causes only a small performance drop (−0.9 F1). The gap between CentralKV (Strict) and H2O remains substantial (+9.5 F1), indicating that the method retains most of its advantage under a matched physical footprint.

K.4 Analysis of Degradation Cases

While CentralKV slightly exceeds the Full Cache macro average overall, the difference is within

Table 11: Detailed LongBench Results (Llama-3-8B, 20% Logical Budget). Gap values are color-coded: positive improvements over Full Cache are green and degradations are red. Statistical significance is reported at the category level in Appendix K; no per-dataset significance is implied.

Category	Dataset	Metric	Full	H2O	GraphKV	CentralKV	Gap
Single-Doc QA	NarrativeQA	F1	23.5	22.5	23.5	23.8	+0.3
	Qasper	F1	35.0	30.0	34.5	35.5	+0.5
	MultiFieldQA-zh	F1	55.2	50.5	54.0	55.5	+0.3
Multi-Doc QA	HotpotQA	F1	41.5	31.0	38.5	40.2	-1.3
	2WikiMultihopQA	F1	39.6	28.0	37.2	38.4	-1.2
	Musique	F1	24.5	17.0	22.5	23.8	-0.7
	MultiFieldQA-en	F1	53.5	34.0	48.5	51.5	-2.0
Summarization	GovReport	Rouge-L	28.0	24.0	27.2	27.5	-0.5
	QMSum	Rouge-L	20.0	19.0	21.5	21.5	+1.5
	MultiNews	Rouge-L	24.0	22.0	24.5	25.0	+1.0
	VCSUM	Rouge-L	14.0	13.0	15.1	15.5	+1.5
Few-Shot	TREC	Acc	65.0	60.0	64.0	65.5	+0.5
	TriviaQA	F1	80.0	77.0	79.5	80.0	0.0
	SAMSum	Rouge-L	38.0	37.0	38.5	39.5	+1.5
Code	LCC	Edit Similarity	55.0	42.5	51.2	58.0	+3.0
	RepoBench-P	Edit Similarity	50.0	40.2	48.5	51.0	+1.0
Synthetic	PassageRetrieval-en	Acc	98.0	82.0	94.0	97.5	-0.5
	PassageRetrieval-zh	Acc	98.0	85.0	95.5	98.0	0.0
Macro Avg. (6 categories; from Table 2)	—	—	51.8	44.0	50.2	52.2	+0.4

Table 12: Strict Physical Budgeting Analysis (2WikiMultihopQA). Green marks our best strict-cap variant; blue marks the strict H2O baseline. CentralKV maintains dominance even when constrained to identical physical memory footprint.

Configuration	Physical Usage	F1 Score	vs. H2O
H2O (Strict Baseline)	20.0% (4GB)	28.0	—
CentralKV (Union/Relaxed)	23.4% (4.7GB)	38.4	+10.4
CentralKV (Strict Cap)	20.0% (4GB)	37.5	+9.5

the reported run-to-run variation and should not be interpreted as compression improving on uncompressed inference. Residual gaps remain on a few datasets, especially in Multi-Doc QA and PassageRetrieval-en. These cases are consistent with settings where the answer depends either on exact retrieval of one buried clue or on preserving multiple dispersed evidential links simultaneously.

- **Observation:** The remaining Multi-Doc QA gap to Full Cache suggests that some samples require preserving several weak but jointly necessary links, not only one dominant bridge token.
- **CentralKV Behavior:** CentralKV substantially narrows this gap relative to H2O and remains strong on code tasks, but the detailed results indicate that lossy cache compression can still underperform Full Cache when evidence is both sparse and distributed.

L Extended Robustness Analysis

L.1 Varying Cache Budgets

We evaluated performance stability across a spectrum of compression ratios (10%–50%).

- **Extreme Compression (10%):** CentralKV retains a substantial fraction of its peak performance. In contrast, H2O and SnapKV drop sharply on Multi-hop QA. This suggests that Structural Bridges capture a compact subset of the context that remains useful under aggressive compression.
- **High Fidelity (50%):** At 50% budget, CentralKV effectively matches the Full Cache baseline (gap < 1.5%), effectively solving the memory wall for practical deployments.

L.2 Model Scalability

To assess generality, the evaluation protocol can be extended to other decoder-only LLMs, including larger model variants, under the same settings.

L.3 Needle In A Haystack (NIAH)

To further probe bridge preservation, we conducted the "Needle In A Haystack" test. In this task, a specific fact (the needle) is inserted into a long context of irrelevant text (the haystack), and the model must retrieve it. The sweep uses Llama-3-8B with a 20% per-head budget, 10 insertion depths, 10 insertion positions, and 100 probes per context length. CentralKV outperforms H2O in this setting, although it remains a lossy compressor and its accuracy also declines as context length grows. This setup is intended to assess whether the mechanism remains effective when the context becomes increasingly sparse and retrieval-sensitive.

The gap over H2O widens from 2 points at 4K to 39 points at 256K. The table reports per-length averages; no depth-by-length heat map is inferred from these aggregate values.

L.4 Counterfactual Causality Test

We assess whether the tokens identified by CentralKV are load-bearing components of the context.

- **Protocol:** On the 2WikiMultihopQA dataset, we identified the "Divergent Set" D —tokens retained by CentralKV but evicted by H2O (Budget=20%).
- **Intervention:** Instead of a soft mask (which alters Softmax normalization), we performed a **Physical Eviction Simulation**. For the Full Cache model, we removed the KV pairs corresponding to tokens in set D from the cache during generation.
- **Outcome:** Removing this subset ($\approx 5\%$ of tokens) caused the F1 score to drop from 35.0 to 24.5 (averaged over 100 randomly selected samples from the validation set).

L.5 Attention Hit Rate Analysis

Do the model's heads actually use the tokens we save? We measured the **Recall of Attention Mass**: the proportion of the Full Cache's total attention mass that falls onto the tokens retained by the compressed cache during the generation phase.

- **Result:** Averaged across the full 2WikiMultihopQA validation set, H2O retains 65% of the attention mass (it keeps frequent tokens, but misses critical sparse dependencies), whereas CentralKV retains **82%** of the attention mass.
- **Implication:** By maximizing the "Graph Centrality," CentralKV implicitly maximizes the

Table 13: Single-needle NIAH accuracy (%; Llama-3-8B, 20% per-head budget; 100 probes per context length).

Context length	Full Cache	H2O	CentralKV
4K	100	97	99
32K	98	74	90
64K	96	58	84
128K	93	42	76
192K	90	30	68
256K	87	22	61

probability that a token will be attended to in the future (Effective Receptive Field coverage).

L.6 Sensitivity to Prompt Templates

LLMs are notoriously sensitive to prompts. To ensure our method is not overfit to LongBench's default templates:

- **Variations:** We tested 3 variants: (1) Standard (LongBench default), (2) Chain-of-Thought ("Let's think step by step..."), and (3) Minimalist (Context + Question only).
- **Result:** While absolute scores shifted, the **Relative Gap** between CentralKV and H2O remained stable (+8% $\sim$ +11%) across all templates. Notably, in CoT prompts, the gap widened slightly (+11.5%), suggesting that CentralKV is particularly adept at preserving the intermediate reasoning steps required for CoT.

M Cross-Model and Reasoning-Benchmark Results

We report the additional evaluations under the same LongBench evaluation protocol and 20% per-head cache budget used for the main comparison. These results broaden the model coverage without changing the main Llama-3-8B-Instruct setting.

For completeness, we also evaluated AIME 2024 as a short symbolic-math diagnostic. On the 30-problem set, the reported pass@1 scores are 10.0 for Full Cache, 3.3 for H2O, and 6.7 for CentralKV. Because these prompts are shorter than the long-context settings and the absolute scores are low, this result primarily reflects the base model's mathematical capability rather than cache retention; we therefore do not use it as evidence of a long-context advantage.

Table 14: LongBench results with Qwen2.5-7B-Instruct (20% per-head budget).

Method	Multi-Doc QA	Code	Summ.	Avg. (6 cat.)
Full Cache	47.5	57.5	26.5	56.2
H2O	36.5	47.2	23.8	50.1
SnapKV	43.2	53.5	25.8	53.8
GraphKV	44.0	53.8	25.5	54.1
CentralKV	45.0	54.3	25.6	54.5

Table 15: LongBench results with DeepSeek-R1-Distill-Qwen-7B (20% per-head budget).

Method	Multi-Doc QA	Code	Summ.	Avg. (6 cat.)
Full Cache	47.0	56.8	26.2	55.6
H2O	38.0	49.5	23.8	51.2
SnapKV	41.8	51.8	25.3	52.4
GraphKV	42.2	52.2	25.0	52.7
CentralKV	42.8	52.5	25.1	53.0

Table 16: LongReason accuracy (%), Llama-3-8B, 20% per-head budget).

Method	Full	H2O	SnapKV	GraphKV	CentralKV
Accuracy (%)	42.0	24.0	33.5	35.5	38.6

N Additional Related Work: System Optimization

Beyond algorithmic compression, system-level optimizations are important for efficient inference. **FlashAttention** (Dao et al., 2022; Dao, 2024) optimizes memory IO to accelerate attention computation. Systems like **DeepSpeed** (Aminabadi et al., 2022) and **FlexGen** (Sheng et al., 2023) improve serving efficiency through kernel/system optimization and offloading on limited GPU memory. To further reduce memory footprint, quantization methods can be combined with cache compression: **KIVI** (Liu et al., 2024b) and the dynamic, importance-aware **KVC-Q** (Wu et al., 2026) directly quantize the KV cache, while **SmoothQuant** (Xiao et al., 2023a) and **AWQ** (Lin et al., 2024) reduce inference memory through weight/activation quantization. Architecturally, **GQA** (Ainslie et al., 2023) reduces the size of the KV cache by sharing keys and values across heads. CentralKV is compatible with these system-level optimizations because it operates at the token selection level.